

2026 Intel Cup Undergraduate Electronic Design Contest
- Embedded System Design Invitational Contest

Project Design Report

Final Report



Intel Cup Embedded System Design Contest

Title: LabGuardian: An Intelligent Teaching Assistant System Based on Edge AI

Student Name: Zhang Xinran, Liu Su'an, Ruan Jiali

Advisor: Cheng Kun

University: Beijing University of Posts and Telecommunications

LabGuardian: An Intelligent Teaching Assistant System Based on Edge AI

ABSTRACT

LabGuardian addresses a common problem in electronics laboratories: a student circuit may match the schematic in intention but be physically miswired on the breadboard. Manual inspection is slow, repetitive, and difficult when one teacher must support many students. The system therefore reconstructs a pin-level electrical netlist from a top-view breadboard image, checks it against a reference circuit with deterministic graph matching, and uses an on-device small language model only to explain the verified result. This separation keeps circuit judgement auditable while still providing student-friendly feedback.

The key design is a neuro-symbolic fact chain: image keypoints identify component pins, breadboard rules map pins to holes and conductive nodes, graph comparison produces structured error codes with evidence, and an evidence-constrained explanation model rewrites those facts into repair guidance. The implementation targets the Intel DK-2500 edge platform. Vision runs on the NPU, explanation on the iGPU, and graph matching, retrieval, telemetry, and orchestration on the CPU, allowing local operation within an 8 GB memory budget.

Key words: breadboard wiring diagnosis, edge AI, neuro-symbolic reasoning, auditable explanation, knowledge distillation

1 Problem and Requirements

Electronics laboratory teaching requires students to translate a schematic into real components, breadboard holes, instrument connections, and measured phenomena. Many failures are not caused by a wrong schematic but by a hidden physical wiring error: a pin is inserted into the adjacent row, a polarized device is reversed, a power rail is missing, or two nets are accidentally shorted. These errors are hard to judge from appearance alone and often require a teacher to check each pin and wire.

Existing approaches are insufficient for this classroom setting. SPICE-style simulation verifies an ideal schematic but cannot see the student's real breadboard. Pure template rules are explainable but fragile when component placement varies. End-to-end multimodal models can describe images but may hallucinate and should not be the sole judge of circuit correctness. LabGuardian instead requires four properties: pin-level perception, deterministic electrical judgement, evidence-linked explanation, and weak-network edge deployment.

The system serves students, teachers, and maintainers. Students need to know what is wrong, why it matters, and how to fix it. Teachers need a fast overview of bench status and repeated mistakes. Maintainers need stable data contracts so that new boards, devices, circuits, and fault cases can be added without rewriting the whole pipeline.

2 System Design

LabGuardian does not ask a large model whether the circuit is right. A diagnosis is divided into three stages. First, visual perception converts the breadboard image into structured physical facts. Second, a deterministic symbolic engine compares those facts with the reference circuit and emits error codes. Third, a constrained teaching model explains only the verified facts. This design makes every judgement traceable to a component, pin, hole, and electrical network.

LabGuardian: Neuro-Symbolic Breadboard Circuit Diagnosis

Perception & netlist reconstruction

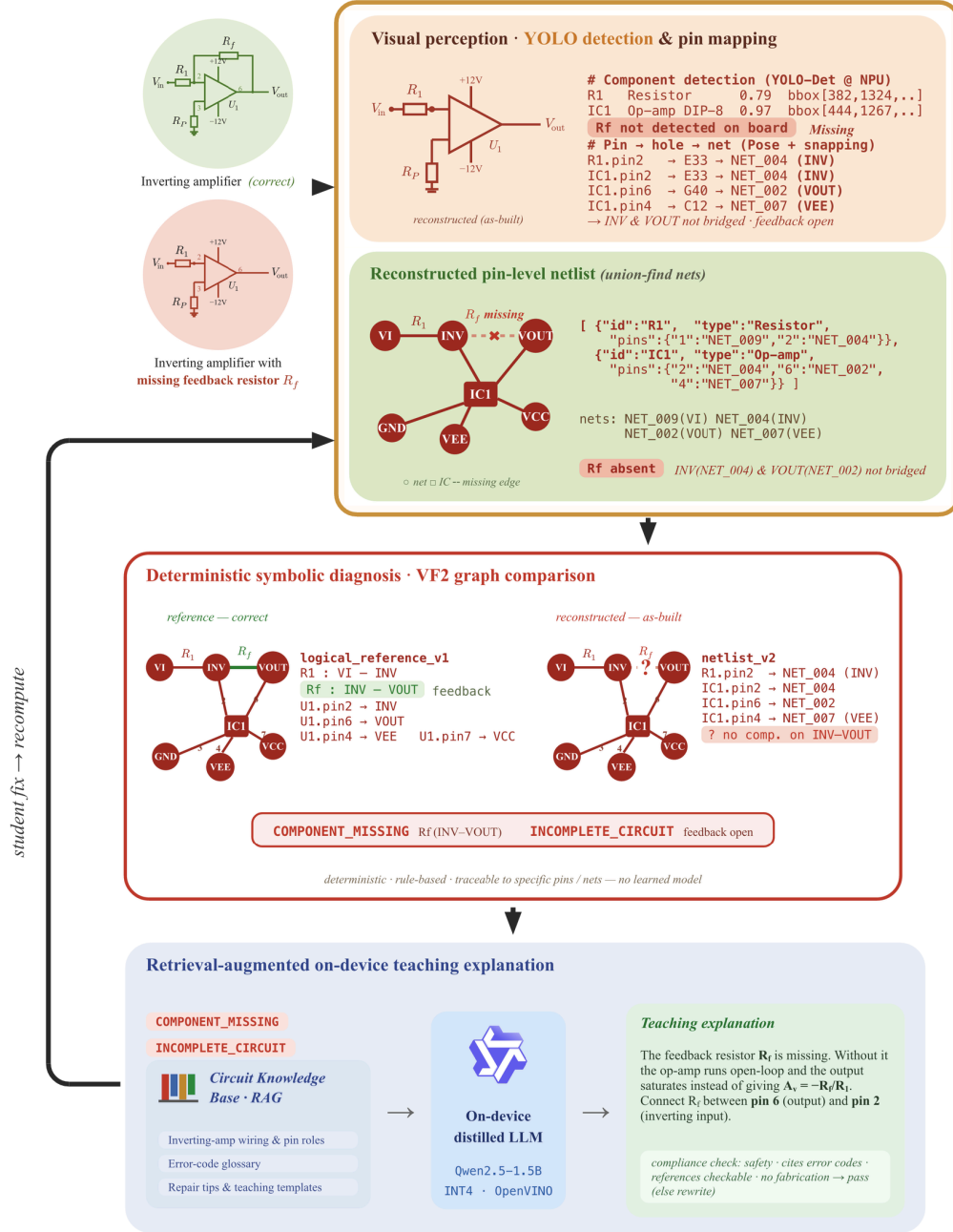


Photo → YOLO netlist → VF2 check vs reference → on-device explanation. The LLM only explains; facts stay auditable.

Fig. 1 Overall neuro-symbolic pipeline. Vision reconstructs a pin-level netlist, deterministic graph matching emits evidence-backed error codes, and the on-device explanation model generates constrained teaching feedback.

The end-to-end data path is: component and pin detection, hole snapping, conductive-node parsing, netlist reconstruction, reference comparison, evidence packaging, explanation generation, and user-interface highlighting. If visual mapping is uncertain, the front end exposes the candidate hole or network to the student. A correction triggers recomputation from topology onward without rerunning vision, which keeps the system usable even when a small number of keypoints are ambiguous.

On the DK-2500 platform, the workload is split by hardware characteristics. YOLO-style pin keypoint detection is dense tensor computation and is assigned to the NPU. Autoregressive explanation generation runs

on the iGPU. Irregular control flow, graph matching, retrieval, telemetry, and API orchestration stay on the CPU. This division avoids resource contention and keeps both vision and explanation local.

3 Visual Perception and Netlist Reconstruction

The visual module uses a self-built breadboard dataset because public circuit datasets rarely combine real classroom photos, pin-level keypoints, hole mapping, and realistic occlusion. The dataset covers typical teaching circuits such as first-order RC, common-emitter amplification, differential pair, and UA741 inverting, integrating, and summing amplifier experiments. Labels combine component boxes with pin keypoints, shifting the recognition target from component-level detection to pin-level electrical evidence.

The trained YOLOv8s-pose model takes a full top-view image instead of cropped component images. Full-image detection preserves context and avoids cutting off pins near component boundaries. Detected keypoints are assigned to component instances using box overlap, center distance, keypoint-in-box ratio, geometry consistency, confidence, and type compatibility. Dense packages such as DIP ICs can use package geometry to infer pins when keypoint detection is unreliable.

A breadboard-specific mapper then converts pin coordinates into physical holes and conductive nodes. It calibrates or synthesizes the breadboard grid, stores candidate holes and confidence, and applies breadboard connectivity rules: terminal strips, center gap isolation, and segmented power rails. Device-level priors, such as two-pin pairing, potentiometer collinearity, and transistor polarity, further reduce visually plausible but electrically impossible assignments.

The final output is a structured netlist: components, pins, physical holes, conductive nodes, and merged electrical networks. Jumper wires are not treated as logical components; they merge equivalent nodes. If a non-wire component has multiple pins on the same network, the system marks the case for short-circuit analysis.

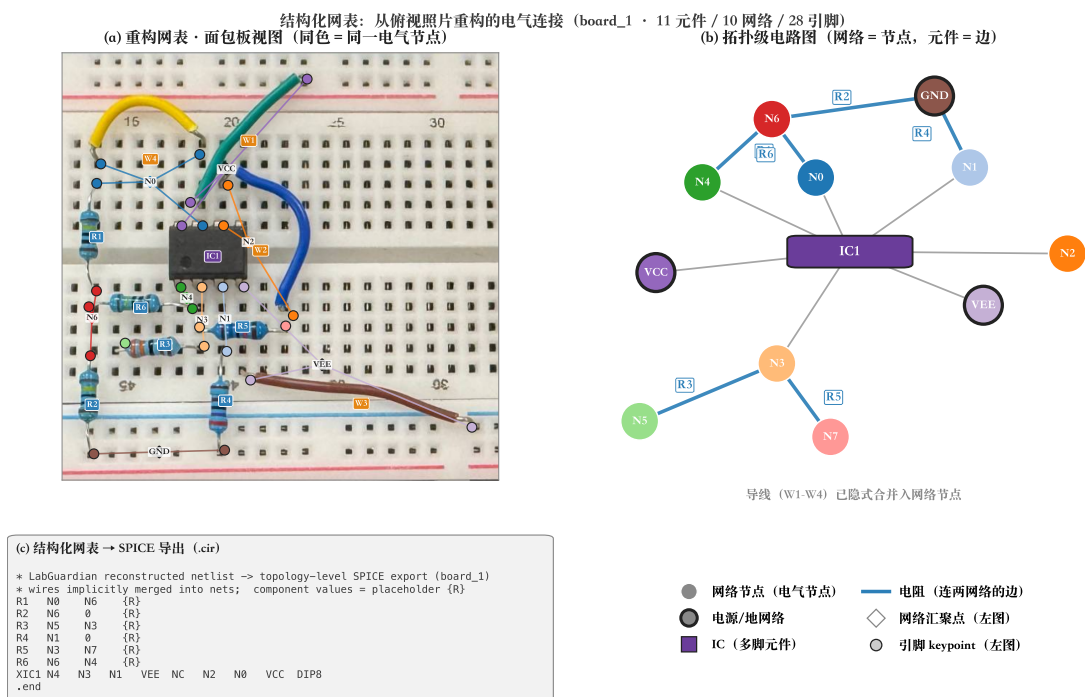


Fig. 2 Structured netlist example: breadboard-level highlighting, topology-level circuit graph, and exported SPICE-like fragment share the same reconstructed facts.

4 Deterministic Diagnosis

The reference circuit is expressed with a small domain-specific language rather than hand-written raw JSON. The DSL declares components, nets, pin roles, power and ground semantics, no-connect pins, symmetric

groups, optional variants, and comparison rules. This keeps author intent readable while producing a normalized reference graph for the checker.

Diagnosis is performed on a component-network bipartite graph. The main checker first tries full graph isomorphism with VF2-style matching, using component type, network role, and pin role constraints. If full matching fails, it falls back to subgraph and containment checks, then to approximate difference reporting. Non-polar two-pin devices can ignore pin order, reasonable extra jumpers can be tolerated, but power, ground, and functional pins remain strict.

The output is not just pass or fail. The checker produces a structured diagnostic report with stable error codes, severity, suggested action, involved components, pins, holes, networks, and evidence references. Typical codes include missing component, wrong connection, short circuit, open circuit, output-node mismatch, critical extra connection, and incomplete circuit. These codes become the bridge between the symbolic checker, the user interface, the retrieval system, and the explanation model.

Table 1 Representative deterministic diagnostic error codes.

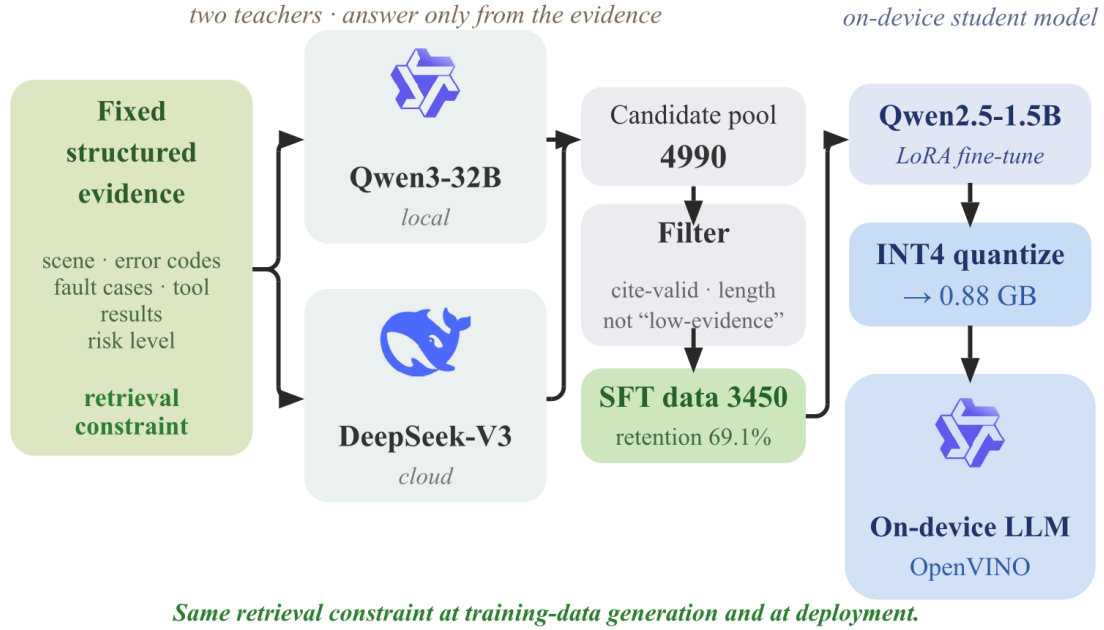
Error Code	Family	Meaning
COMPONENT_MISSING	Missing	A required reference component is absent.
WRONG_CONNECTION	Wiring	A pin is connected to a network inconsistent with the reference.
SHORT_CIRCUIT	Short	Critical nets such as power and ground are merged.
OPEN_CIRCUIT	Open	Pins that should share a net are separated.
OUTPUT_NODE_MISMATCH	Node	The output role is assigned to the wrong network.
INCOMPLETE_CIRCUIT	Partial	Only part of the reference circuit is present.

5 Evidence-Constrained Explanation

After deterministic diagnosis, the language model is allowed to explain but not to rejudge. LabGuardian builds a compact fixed-evidence context containing the scene, current error codes, involved components and pins, risk level, allowed tools, and retrieved teaching knowledge. The model can only use this injected evidence. A consistency checker rejects answers that omit required evidence, invent devices or pins, ignore dangerous power cases, or contradict the diagnostic report.

Retrieval is deliberately narrow. Valid channels are the teaching-scenario library, fault-case library, device datasheet library, circuit-knowledge library, and structured runtime facts. Fault cases are recalled by the current error code, and device datasheets are filtered by the active experiment. If retrieval fails, the system fails closed and falls back to deterministic templates rather than inventing missing knowledge.

The on-device explanation model is trained by response distillation under the same fixed-evidence constraint used at deployment. Local Qwen3-32B and cloud DeepSeek-V3 teacher responses are generated for the same evidence packages, then filtered for citation compliance, length, safety, and consistency. The retained 3450 supervised fine-tuning samples train a Qwen2.5-1.5B-Instruct student with LoRA. After merging LoRA weights, the model is quantized to INT4 and exported to OpenVINO for DK-2500 deployment.



Data distillation: supervised fine-tune on grounded, filtered teacher answers — no logits / KL alignment.

Fig. 3 Fixed-evidence distillation and deployment loop. Teachers answer under the same evidence boundary as the edge student model, and invalid answers are filtered before training.

Board-side measurements show practical edge feasibility. The INT4 explanation model package is about 941.5 MB, loads in about 3.74 s, has about 63.9 ms first-token latency, decodes at about 23.2 tokens/s, and uses about 1.36 GB peak memory. A 30-question preliminary quality check achieved 90.0% overall pass rate, with diagnostic questions remaining the main area for improvement.

6 Results and Conclusion

The key result is not a single model score but a complete auditable loop from real image to deterministic diagnosis to constrained explanation. On the vision side, DK-2500 NPU INT8 inference for YOLOv8s-pose reaches 13.37 ms average latency, 15.61 ms P99 latency, and 74.7 images/s. On the explanation side, the INT4 student model runs locally within the edge memory budget. On the reasoning side, circuit correctness is decided by graph matching and stable error codes rather than free-form model judgement.

Table 2 Key quantitative evidence retained from the Chinese report.

Aspect	Metric	Result
Vision	NPU INT8 YOLOv8s-pose	13.37 ms average, 74.7 images/s, P99 15.61 ms
Energy	NPU INT8 incremental power	About 4.12 W and 114.2 mJ per inference
Explanation	INT4 student model	About 0.94 GB package and 1.36 GB peak memory
Quality	30-question check	90.0% overall pass rate; diagnostic guidance needs further work
Reliability	Evidence boundary	Model explains only verified facts; unsupported claims are blocked or templated

LabGuardian demonstrates that classroom breadboard diagnosis can be made both intelligent and auditable. Neural perception handles the difficult image task, symbolic graph matching decides correctness, and a small edge model provides readable teaching feedback within fixed evidence boundaries. The current limitations are also clear: larger real-board end-to-end testing is still needed, diagnostic explanations should become more pedagogically guided, and the scenario and fault libraries should be expanded to cover more laboratory courses.